

B. The Universal Layout Controller (**MainActivity.kt**)

Replace the contents of your project's MainActivity.kt file with the script package detailed below.

To alternate between local files and live website links, you only need to change the string assigned to the **TARGET_URL** variable at the top of the file.

Kotlin Code

```
package com.example.jsonextractor02
```

```
import android.annotation.SuppressLint
import android.os.Bundle
import android.webkit.WebChromeClient
import android.webkit.WebSettings
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.viewinterop.AndroidView
import com.example.jsonextractor02.ui.theme.JsonExtractor02Theme
```

```
class MainActivity : ComponentActivity() {
```

```
    //
```

```
=====
```

```
=
```

```
    // UNIVERSAL CONTROL SWITCH
```

```
    // Set this parameter to your local file path or any active website URL!
```

```
    //
```

```
=====
```

```
=
```

```
    private val TARGET_URL = "file:///android_asset/json-path-extractor.html"
```

```

// Example alternative configuration for a public web URL:
// private val TARGET_URL = "[https://www.google.com](https://www.google.com)"

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        JsonExtractor02Theme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                UniversalWebView(
                    url = TARGET_URL,
                    modifier = Modifier.padding(innerPadding)
                )
            }
        }
    }
}

@SuppressLint("SetJavaScriptEnabled")
@Composable
fun UniversalWebView(url: String, modifier: Modifier = Modifier) {
    AndroidView(
        modifier = modifier.fillMaxSize(),
        factory = { context ->
            WebView(context).apply {
                // 1. Force navigation clicks and web redirect events to stay inside the app
                webViewClient = WebViewClient()
                webChromeClient = WebChromeClient()

                // 2. Adjust browser performance engine settings
                settings.apply {
                    javaScriptEnabled = true    // Allows Javascript processing logic
                    domStorageEnabled = true    // Enables theme tracking and configuration
                    databaseEnabled = true      // Activates client-side data queries

                    // Viewport and responsive layout computation
                    useWideViewPort = true
                    loadWithOverviewMode = true

                    // Local storage directory file access credentials
                    allowFileAccess = true
                }
            }
        }
    )
}

```

```

        allowContentAccess = true

        @Suppress("DEPRECATION")
        allowUniversalAccessFromFileURLs = true
        @Suppress("DEPRECATION")
        allowFileAccessFromFileURLs = true

        // Allow local offline assets to fetch online fonts and dependency tools
securely
        mixedContentMode = WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
    }

    // 3. Prevent structural elements from collapsing into zero pixels
    layoutParams = android.view.ViewGroup.LayoutParams(
        android.view.ViewGroup.LayoutParams.MATCH_PARENT,
        android.view.ViewGroup.LayoutParams.MATCH_PARENT
    )

    // 4. Fire up the page pipeline
    loadUrl(url)
    }
    }
    )
}

```

4. Architectural Blueprint for Fresh Application Development

When you create a brand-new application project inside Android Studio in the future, follow this sequence:

Step 1: Project Generation

1. Launch Android Studio, click New Project.
2. Select Empty Activity (make sure it shows the multi-colored Compose icon).
3. Assign your project a name, choose Kotlin as your programming language, and select your Minimum SDK option. Click Finish.

Step 2: Inject Security Permissions

1. Navigate through your project tree to app > manifests > AndroidManifest.xml.

2. Put the network permission token code right above the main opening `<application>` marker element:

XML Code

```
<uses-permission android:name="android.permission.INTERNET" />
```

Step 3: Bundle Your Web App Code Assets (For Offline Projects)

1. In the file explorer tab on the left, change the top selector dropdown to Android.
2. Right-click your main app package component folder.
3. Choose New > Folder > Assets Folder. Click Finish.
4. Move your custom target webpage configuration files (like `json-path-extractor.html`, code styles, or asset layouts) straight into this new folder.

Step 4: Inject the Controller Module

1. Navigate to app > java > your.package.name > MainActivity.kt.
2. Clean out the default starter text template code entirely.
3. Paste the provided Universal MainActivity.kt source solution directly into the document.
4. Update the assignment value link defined under `TARGET_URL`:
 - For Offline Files: `"file:///android_asset/yourfile.html"`
 - For Online Domains: `"https://your-domain.com"`

Step 5: Match Application Code Styles

When a fresh project file generates, your device defines a unique style wrapper configuration derived from your chosen app title. Look closely at lines 28 & 30 of the code above.

Replace `JsonExtractor02Theme` with the theme function generated by Android Studio for your new project (found inside your project package directory at `ui.theme`). For example, if your new project is called "DataViewer", change it to `DataViewerTheme`.

5. Summary Checklist for Success

- [x] Network Permission Added: `android.permission.INTERNET` is declared in the manifest.
- [x] JavaScript Activated: `settings.javascriptEnabled = true` is explicitly configured.
- [x] Storage Systems Activated: Both `domStorageEnabled` and `databaseEnabled` are enabled.
- [x] Mixed Content Enabled: `MIXED_CONTENT_ALWAYS_ALLOW` bridges local file code components with external URL script dependencies safely.
- [x] Layout Sizing Locked: Layout parameters are forced to match the device screen dimensions (`MATCH_PARENT`) to guarantee responsive flex renders correctly. """"